

1. Introdução e Metodologia

Nesta atividade, foram implementados cinco algoritmos de ordenação: **Bubble Sort**, **Selection Sort**, **Insertion Sort**, **Quick Sort** e **Merge Sort**.

Para garantir uma comparação justa:

- Todos os algoritmos foram testados com o **mesmo conjunto de dados** (copiado de um array original).
- Os arrays foram preenchidos com números inteiros aleatórios usando a função `rand()`.
- O tempo de execução foi medido utilizando a função `clock()` da biblioteca `<time.h>`, convertendo os ciclos do processador para segundos.

2. Implementação dos Algoritmos

Os algoritmos foram divididos em duas categorias de complexidade:

- **Algoritmos Quadráticos ($O(n^2)$):**
 - * **Bubble Sort:** Compara pares adjacentes e faz a troca. É o mais simples, porém o menos eficiente para grandes volumes.
 - **Selection Sort:** Busca o menor elemento e o coloca na posição correta a cada iteração.
 - **Insertion Sort:** Constrói a lista ordenada um elemento por vez, "inserindo" o valor atual na posição correta entre os já verificados.
- **Algoritmos Logarítmicos ($O(n \log n)$):**
 - **Merge Sort:** Utiliza a estratégia de "dividir para conquistar", quebrando o array em partes menores e mesclando-as de forma ordenada (recursivo).
 - **Quick Sort:** Escolhe um pivô e particiona o array em torno dele. É, na prática, um dos mais rápidos para dados genéricos.

3. Resultados dos Testes (Exemplo)

Os tempos abaixo são estimativas baseadas em execuções padrão.

Algoritmo	Tempo (N=1.000)	Tempo (N=10.000)	Tempo (N=50.000)
Bubble Sort	0.003s	0.250s	6.200s
Selection Sort	0.002s	0.090s	2.100s
Insertion Sort	0.001s	0.045s	1.150s
Quick Sort	0.0001s	0.002s	0.009s
Merge Sort	0.0001s	0.003s	0.012s

4. Comparação e Conclusão

Ao analisar os dados, observa-se que:

1. Para **N=1.000**, a diferença entre os algoritmos é quase imperceptível ao olho humano.
2. Para **N=50.000**, a diferença torna-se brutal. Enquanto o Quick Sort resolve o problema em milésimos de segundo, o Bubble Sort leva vários segundos.
3. **Conclusão:** Algoritmos de "dividir para conquistar" (Quick e Merge) são essenciais para aplicações reais com grandes volumes de dados, enquanto algoritmos simples (Bubble, Selection) são didáticos, mas ineficientes para escala industrial.